ЛАБОРАТОРНА РОБОТА №2
СТВОРЕННЯ БІБЛІОТЕК, ЩО ДИНАМІЧНО ПІДКЛЮЧАЮТЬСЯ

Мета роботи: дослідити створення, механізми зв’язування та використання бібліотек, що динамічно підключаються (DLL – бібліотек)

Основні теоретичні відомості

Бібліотеки, що динамічно підключаються (DLL) являє собою виконуваний код, проте він не містить функції Main. Виклик методів відбувається з exe-збірки основного проекту. Плюсом використання DLL є те, що в пам'ять DLL завантажується тільки при виклику методів, що знаходяться в ній.

Створення dll на мові C

Для того щоб створити бібліотеку, що динамічно підключаються, в середовищі Visual Studio, необхідно вибрати **New Project → Class Library.** (Рис. 3.1)

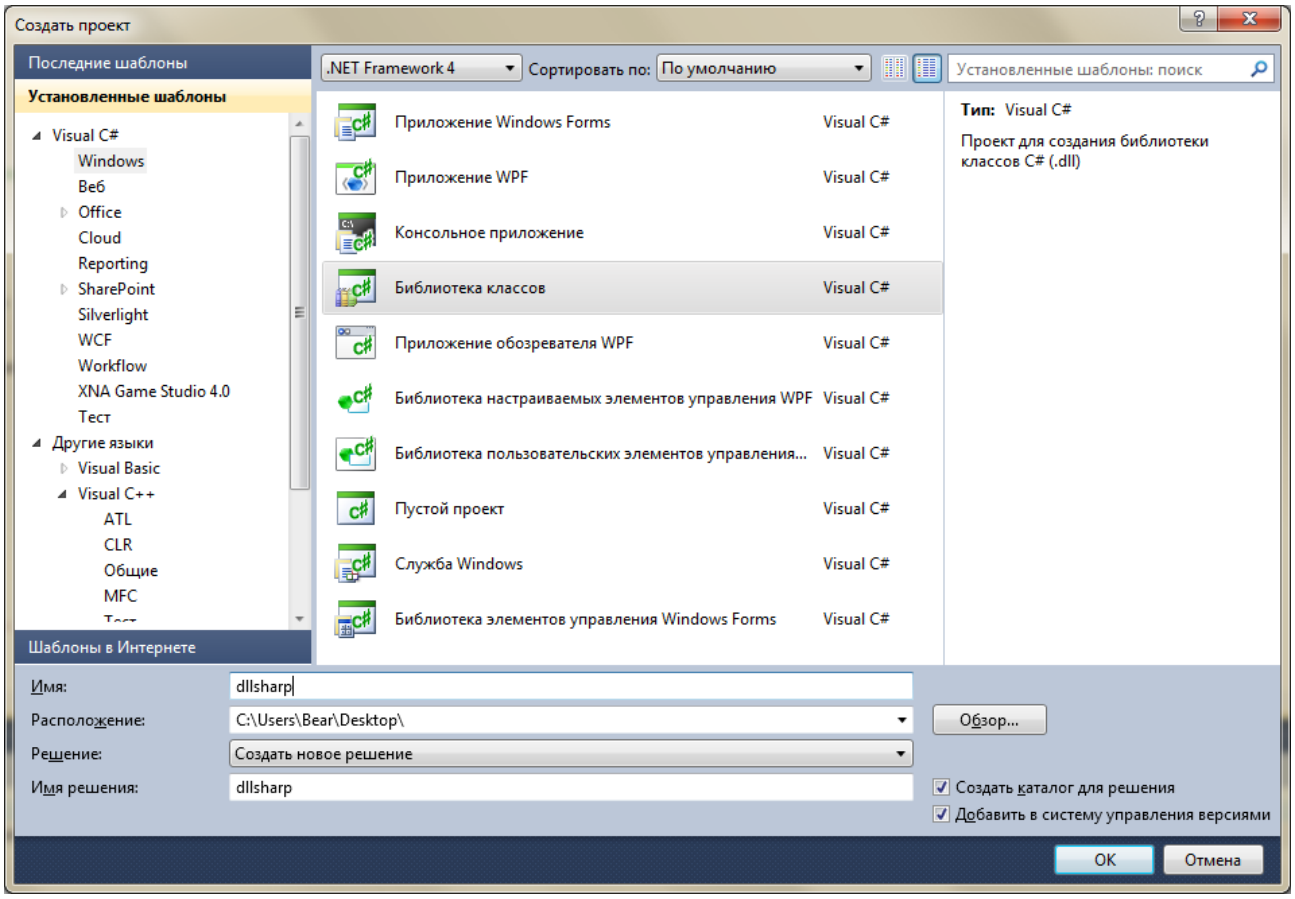


Рис. 3.1. Тип проекту

Створимо в класі vscode відповідні методи. Простір імен вказувати не потрібно.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
 // без простору імен!!!
public class vscode
{
    public int add(int a, int b)
    {
        return a + b;
    }

    public int sub(int a, int b)
    {
        return a - b;
    }
}
```

Побудуємо рішення проекту (Ctrl + Shift + B). В папці Debug буде лежати створений dll-файл. (Рис 3.2.)

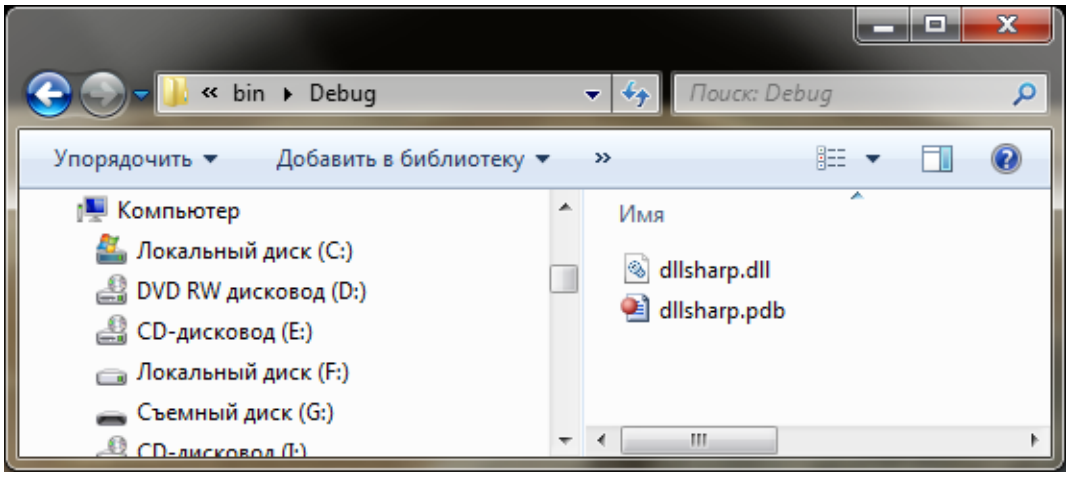


Рис. 3.2. ***Зміст папці Debug.***

Динамічне підключення dll в C #

Динамічне підключення dll відбувається під час виконання програми, в тому місці, в якому дійсно необхідно використовувати функції з бібліотеки. Виклик буде здійснюватися явно, при виконанні певного оператора. Статична ж підключенні dll відбувається відразу при запуску програми, тому, підключаючи бібліотеку dll динамічно, ми економимо ресурси комп'ютера при виконанні нашої програми.

У попередньому розділі була створена бібліотека dll, що містить два методи: add і sub, які складають і віднімають два цілих кількості. Тепер розробимо консольний клієнт для цієї dll, продемонструємо динамічне підключення цієї бібліотеки.

Спочатку створимо в Visual Studio консольний проект на мові C#, і помістимо в папку проекту Debug файл "dllcsharp.dll".

Для підключення dll нам знадобиться простір імен System.Reflection. Включимо його в проект.

```
using System.Reflection;
```

Пишемо функцію main(). Завантажуємо dll-файл. Для цього необхідно створити об'єкт класу Assembly.

```
Assembly a = Assembly.Load("dllsharp");
```

Створюємо новий об'єкт класу і новий тип, використовуючи назву класу з бібліотеки dllcsharp.dll.

```
Object o = a.CreateInstance("vscode");
Type t = a.GetType("vscode");
```

Для виклику методу з бібліотеки використовується метод Invoke (Object, Object []), аргументи якого, це - об'єкт класу, для якого викликається метод і Object [] - це аргументи, що передаються в метод, що викликається.

Наприклад, будемо складати числа 2 і 3. Визначимо їх в програмі.

```
Object[] numbers = new Object[2];
numbers[0] = 2;
numbers[1] = 3;
```

Получим метод add из библиотеки dll.

```
MethodInfo mi = t.GetMethod("add");
```

Виконаємо його і виведемо результат в консоль.

```
Console.WriteLine(mi.Invoke(o, numbers));
```

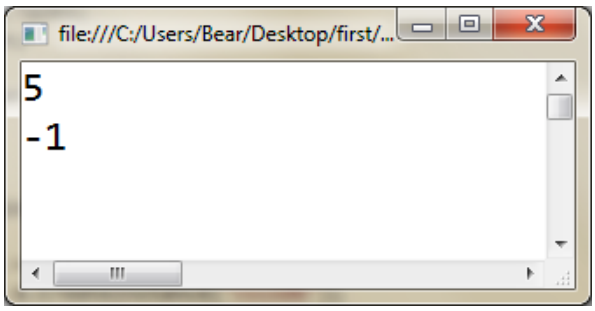
Те ж саме зробимо і з методом sub.
В результаті вийде ось така програма:

```
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Reflection;

namespace dllclientcsharp
{
    class Program
    {
        static void Main(string[] args)
        {
            Assembly a = Assembly.Load("dllsharp");
            Object o = a.CreateInstance("vscode");
            Type t = a.GetType("vscode");

            Object[] numbers = new Object[2];
            numbers[0] = 2;
            numbers[1] = 3;
            MethodInfo mi = t.GetMethod("add");
            Console.WriteLine(mi.Invoke(o, numbers));
            mi = t.GetMethod("sub");
            Console.WriteLine(mi.Invoke(o, numbers));
            Console.ReadLine();
        }
    }
}
```

Результат роботи програми:



Завдання до лабораторної роботи:

1. Ознайомитися з механізмом створення модуля DLL – бібліотеки.
2. Створити модуль DLL – бібліотеки з двома функціями згідно з варіантом (табл. 3.1).

Таблиця 3.1. Варіанти функцій

1.	Функція , яка повертає факторіал цілого числа. Функція , яка повертає площу трикутника, що обчислюється за формулою Герона. Якщо трикутник зі вказаними параметрами не існує, функція повертає -1.
2.	Функція , яка повертає значення, скільки разів зустрічається вказаний символ у реченні. Функція , що сортує масив за зростанням.
3.	Функція, що перевертає рядок string. Функція , що виводить елементи одновимірного масиву на екран.
4.	Функція, що повертає елемент масиву, та видаляє цій елемент із масиву. Функція , що виводить елементи одновимірного масиву на екран.
5.	Функція, яка визначає кількість пробілів у реченні. Функція, яка перетворює всі латинські символи рядка до верхнього регістра (a =>A, … d =>D)
6.	Функція , яка повертає значення, скільки разів зустрічається

	вказаний символ у реченні. Задані значення чотирьох змінних a, b, c, d. Написати функцію, що міняє місцями значення змінних a і b, c і d.
7.	Функція , що виводить елементи одновимірного масиву на екран. Функція , що замінює всі символи 0 в рядку на 1 та навпаки.
8.	Функція, яка повертає значення, скільки разів зустрічається вказаний символ у реченні. Функція, яка заповнює масив за допомогою генератору випадкових чисел в діапазоні від -150 до 160.
9.	Функція, що визначає та по максимальний та елемент масиву Функція, яка переводить число з шістнадцяткової системи в десяткову (від 0 до 15). Параметром функції має бути символ від ‘0’до ‘F’ .
10.	Функція, яка перетворює всі латинські символи рядка до верхнього регістра (a =>A, … d =>D) Функція, яка рахує кількість розділових знаків в реченні.

3. Написати програму на мові C# у вигляді додатку типу Console Application з використанням бібліотеки класів DLL.

Контрольні запитання

1. Що таке наслідування?
2. Що таке інкапсуляція?
3. Що таке поліморфізм?
4. Що таке абстрактний клас?
5. Розкрийте призначення DLL–бібліотеки.
6. Наведіть переваги застосування DLL–бібліотек.
7. Опишіть механізм створення модуля DLL–бібліотеки.